A Resource-Aware Forecasting Architecture for Decision Support Systems: Multi-Objective Pruning of Ensemble and Deep Models for Option Pricing

Rui Zhang[a], Jiayi Li[b], Binglin Guo[a] and Chenxue Yang[c,*]

[a] School of Management, Beijing Institute of Technology, Beijing, 100190, China
[b] Shenzhen International Graduate School, Tsinghua University, Shenzhen, 518055, China
[c] Agricultural Information Institute, Chinese Academy of Agricultural Sciences, Beijing, 100081, China

ARTICLE INFO

Keywords:
Decision support systems
information systems architecture
resource-aware forecasting
multi-objective optimization
model pruning
option pricing

ABSTRACT

Accurate and low-latency forecasting is a core capability of decision-support information systems (DSS), yet it remains challenging when the underlying data are high-dimensional, noisy, and nonstationary. This paper proposes a resource-aware forecasting architecture that integrates multi-objective learning into an end-to-end prediction service, and evaluates its effectiveness using option pricing as a representative benchmark. We develop and compare five forecasting backbones—multi-grained cascade forest (gcForest), recurrent neural networks (RNN), long short-term memory (LSTM), Transformer, and vector-quantized variational autoencoder (VQ-VAE)—and enhance them with a three-objective optimization framework (TOOCEP) to jointly balance forecasting accuracy, inference latency, and model size for deployable decision-support scenarios. In addition, an R-measure-guided feature evaluation mechanism is incorporated to refine the input set and improve robustness. Experiments on S&P 500 Index Options (1998–2021), consisting of 11 option-specific features and 7 macroeconomic indicators, apply standardized preprocessing (normalization, missing-value imputation, and outlier handling) and report both predictive and system-level metrics. Results show that the TOOCEP-optimized gcForest achieves the best overall performance (99.78% accuracy) with efficient inference, while neural baselines exhibit clear accuracy–efficiency trade-offs. These findings provide practical guidance for integrating forecasting components into DSS/BI systems under resource constraints and demonstrate a reusable architecture for deploying learning-based time-series forecasters.

1. Introduction

Decision-support and business-intelligence (DSS/BI) systems increasingly rely on forecasting modules that can be deployed as online services, where accuracy must be balanced against inference latency and resource usage (e.g., model size and compute cost). Option price prediction provides a representative high-dimensional, stochastic time-series use case for such deployable forecasting services. In a recent reporting period, the global trading volume of exchange-traded derivatives—including options and futures—reached an unprecedented 200.9 billion contracts, representing a 48.8% year-over-year increase [1]. This expansion highlights both the operational demand for timely predictive analytics and the engineering challenge of delivering low-latency forecasts under resource constraints.

Traditional option pricing approaches, such as the Black–Scholes model for European options and the Binomial Lattice method for American options [2], often struggle to reflect real-world market dynamics because they rest on simplifying assumptions (e.g., constant volatility and frictionless trading). These limitations motivate data-driven learning methods that can exploit large-scale, high-dimensional signals and capture nonlinear dependencies, making them attractive for integration into decision-support pipelines.

Recent advances in learning-based forecasting provide a wide design space for building such modules. Ensemble methods such as the multi-grained cascade forest (gcForest) offer robust feature composition and generalization with moderate training cost, while deep neural architectures—including Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), Transformer models, and Vector Quantized Variational Autoencoders (VQ-VAE)—have shown promise in modeling long- and short-range temporal dependencies. However, when these models are embedded into DSS/BI systems, they frequently face practical obstacles including overfitting, high computational overhead, and inefficient feature utilization, which can hinder stable deployment and scalable maintenance [3–5].

To address these issues from a system-deployability perspective, this study proposes a resource-aware forecasting module that enhances five predictors—gcForest, RNN, LSTM, Transformer, and VQ-VAE—via a three-objective optimization framework (TOOCEP) for model pruning. The optimization targets a deployable balance among predictive accuracy, inference efficiency, and model compactness, aligning model selection with operational constraints in decision-support settings. In addition, an R-measure-based feature selection mechanism is incorporated to refine input variables, improve robustness, and facilitate integration with

*Corresponding author.
1120220789@bit.edu.cn (R. Zhang); lijiaiyi25@mails.tsinghua.edu.cn (J. Li); 1120230473@bit.edu.cn (B. Guo); yangchexue@caas.cn (C. Yang)
ORCID(s):

feature stores commonly used in information-system architectures. The proposed module is validated on Standard & Poor's 500 Index Options from 1998 to 2021, including 11 option-specific characteristics and 7 macroeconomic indicators.

The contributions of this study are fourfold:

1. We formulate option pricing prediction as a deployable forecasting module within DSS/BI architectures and conduct a unified comparison of five representative ensemble and deep models.
2. We introduce a TOOCEP-based tri-objective pruning strategy that improves efficiency and generalization, supporting resource-aware deployment.
3. We integrate R-measure-driven feature selection to strengthen robustness and reduce unnecessary input complexity for operational pipelines.
4. We validate the framework on a large-scale real-world dataset and discuss its practical implications for maintainable and scalable decision-support forecasting.

The remainder of this paper is organized as follows: Section 2 reviews related work, Section 3 presents the proposed methodology and optimization framework, Section 4 reports experimental results and comparative analysis, and Section 5 concludes with findings and future directions.

2. Related Work

Forecasting in financial domains has been widely studied, yet decision-support deployments introduce additional requirements—such as predictable latency, model compactness, and maintainability—that are often under-emphasized in purely accuracy-driven studies. Existing work spans conventional machine learning, deep neural forecasting, and Transformer-based sequence models, providing useful baselines but leaving room for systematic resource-aware, multi-objective modeling in option pricing.

Conventional machine learning and feature-driven forecasting. Classical learning techniques such as Self-Organizing Maps (SOM), Support Vector Regression (SVR), and Support Vector Machines (SVM) have been used to model correlations in large stock datasets and can perform competitively when feature engineering is effective [6]. In practice-oriented pipelines, candlestick patterns, fundamentals, and time-series components are integrated to improve prediction across markets [7]. Related research also explores predictive classification in financial decision contexts; for example, deep learning has been used to classify financial health status as part of broader assessment workflows [8]. While these approaches offer interpretability and simplicity, they may not fully capture nonlinear temporal dynamics or scale well when the feature space grows.

Deep learning for time-series dependencies and robust representations. Deep architectures have advanced forecasting performance by learning temporal representations from multi-feature inputs. CNN variants and hybrid designs have been adopted for end-to-end prediction tasks, and LSTM-based models remain a common baseline for capturing temporal dependencies [9, 10]. Model design has also evolved toward optimization-driven intelligent predictors and comparative evaluations under market stress, highlighting that no single architecture dominates across regimes [11]. Variational Autoencoders (VAEs) further enable probabilistic representation learning for forecasting on financial datasets [12], and transfer-learning-enhanced neural approaches have been explored to improve forecasting stability in shifting environments [6, 13]. Additionally, sentiment and multi-source data integration has been used to augment sequence models, though such designs may increase system complexity and operational burden [9, 10, 14].

Optimization, hybrid modeling, and scalable Transformer forecasting. Hybrid pipelines combining signal processing and optimization strategies are frequently reported as effective ways to reduce prediction error, especially when data are noisy and nonstationary [15]. Swarm and heuristic optimization have also been used to tune sequence models, such as LSTM–PSO hybrids for stock index forecasting [11, 16]. Transformer architectures, with attention-based sequence modeling, have shown strong potential in global index prediction and multi-source fusion settings [17]. Recent work explores Transformer variants to capture inter-asset relationships [4] and introduces specialized forecasting models such as NHITS for volatility prediction [13]. Interpretable multi-view deep architectures and hybrid fusion models have also been proposed to mine long-term dependencies and improve forecasting accuracy [8, 10, 18].

Gaps and motivation. Despite these advances, three gaps remain salient for information-system deployment. First, many studies prioritize accuracy without explicitly optimizing accuracy–latency–model size trade-offs, making it difficult to select models under operational constraints. Second, feature utilization is often not systematically refined for production pipelines, resulting in redundant inputs and fragile generalization. Third, most comparative work focuses on stock forecasting, while option pricing—with distinct microstructure and feature composition—remains less explored under a unified, deployability-oriented protocol. These gaps motivate resource-aware, multi-objective forecasting frameworks that can be integrated into DSS/BI architectures and evaluated with both predictive and system-level metrics.

3. Method

3.1. System Pipeline and Module Interfaces

We design a resource-aware forecasting architecture that can be integrated as a pluggable component in decision-support information systems (DSS/BI). The architecture is organized as an end-to-end pipeline with explicit interfaces for data ingestion, feature store, model training/pruning, model registry, and inference service. Option pricing is treated as a representative high-dimensional time-series application, while the pipeline is generic enough to be reused for other forecasting tasks.

Pipeline stages (online + offline):

- (S1) Data ingestion and normalization. Raw option data and macro indicators are collected and aligned by timestamp. Standard preprocessing is applied (normalization, missing-value imputation, outlier handling).
- (S2) Feature store and feature refinement. We build a feature store that contains: (i) option-specific characteristics, (ii) macroeconomic indicators, and (iii) theory-guided R-factors derived from the R-measure (Section 3.2). Feature refinement is performed using R-measure-driven feature evaluation to reduce non-critical information and improve robustness [19].
- (S3) Model training and resource-aware optimization. Candidate backbones (gcForest, RNN/LSTM, Transformer, VQ-VAE) are trained. A tri-objective pruning framework (TOOCEP/TOOSLP) is then applied to obtain Pareto-efficient model variants under deployment constraints (Section 3.5).
- (S4) Model registry and deployment. The selected model snapshot is versioned in a registry with metadata (training window, feature schema, pruning setting, latency/size profile).
- (S5) Inference service API. The deployed module exposes a standard interface: feature vector x_t at time t: concatenation of $[x_t^{\text{opt}}, x_t^{\text{macro}}, x_t^{Rf}]$; optional: risk context c_t from the Bayesian risk layer (Section 3.3). Forecasted option price $\hat{y}_{t+\Delta}$ or return $\hat{r}_{t+\Delta}$. Optional: uncertainty/risk level (if the Bayesian layer is enabled).

This explicit modularization improves maintainability (components can be updated independently), extensibility (new backbones or features can be plugged in), and deployability (latency/size constraints are treated as first-class objectives).

3.2. R-measure Predictive Framework and R-factor Construction

Option data often contain a large amount of non-critical information, yielding a low signal-to-noise ratio [20]. To reduce overfitting risk and improve interpretability, we inject prior economic structure by integrating asset-pricing theory into the learning pipeline via the R-measure [21, 22]. The key idea is to compute R-factors as theory-guided variables, and then concatenate them to the data-driven feature set.

3.2.1. Black-Scholes-Merton expectation under the R-measure

We start from the Black–Scholes–Merton (BS) setting and express the expected index price under the R-measure [20]:

$$\mathbb{E}_t[S_H] = S_t e^{u(H-t)} = S_t e^{(1+f_3(X_t)+r_t)} \tag{1}$$

The R-measure provides a unified way to compute expectations for mid-horizon derivatives. Let the state dynamics under the R-measure be:

$$\begin{aligned} dY_s = &(r - q - \frac{v_s}{2} + I_{\{s<H\}} v^S v_s + I_{\{s<H\}} v^J) ds \\ &+ \sqrt{v_s}(\rho\, dW_s^R + \sqrt{1-\rho^2}\, dW_{2s}^R) \\ &+ d\left(\sum_{i=1}^{N_s} J_{S,i}\right) - \mathbf{1}_{\{s<H\}} \lambda \bar{\mu} ds - \mathbf{1}_{\{s\geq H\}} \lambda^* \bar{\mu}^* ds, \end{aligned} \tag{2}$$

and

$$dv_s = [\alpha_v(m_v - v_s) - \mathbf{1}_{\{s\geq H\}} \gamma^v v_s] ds + \sigma_v \sqrt{v_s} dW_{1s}^R + d\left(\sum_{i=1}^{N_s} J_{v,i}\right). \tag{3}$$

3.2.2. Measure transformation and mid-horizon pricing (Merton-style)

Assume the holding horizon H ($\leq T$). The Brownian motion under the equivalent expected measure $\mathcal{R}$ satisfies [23]:

$$W_s^R = W_s^P + \int_0^s \gamma\, I_{\{u\geq b\}}\, du \tag{4}$$

where γ is the market price of risk and $\mathbf{1}_{\{\cdot\}}$ is the indicator function.

Then the dynamics of $(A_t)_{t\geq 0}$ under $\mathcal{R}$ are:

$$\frac{dA_s}{A_s} = (r + \sigma_A \gamma \mathbf{1}_{\{s<b\}})\, ds + \sigma_A\, dW_s^R = (\mu \mathbf{1}_{\{s<b\}} + r \mathbf{1}_{\{s\geq b\}})\, ds + \sigma_A\, dW_s^R. \tag{5}$$

The time-t expectation of the option payoff at $t+\tau$ is:

$$\mathbb{E}_t^P[D_{t+\tau}^T J] = \mathbb{E}_t^R[e^{-r(\tau-b)} D_{t+\tau}^0 J] = \mathbb{E}_t^R\left[e^{-r(\tau-b)} \min\left(F, A_{t+\tau}\right)\right]. \tag{6}$$

Since $A_{t+\tau}$ is lognormal under $\mathcal{R}$, the mid-horizon option price becomes:

$$\mathbb{E}_t^P[D_{t+\tau}^T J] = F e^{-r(\tau-b)} N(d_2^b) + A_t e^{ib} N(-d_1^b), \tag{7}$$

with

$$d_1^b = \frac{\ln\left(A_t/F\right) + \mu b + r(\tau - b) + \frac{1}{2}\sigma_A^2 \tau}{\sigma_A \sqrt{\tau}}, \quad d_2^b = d_1^b - \sigma_A \sqrt{\tau}. \tag{8}$$

3.2.3. R-factors and transfer formulation

From the above, we define two R-factors used in our feature store: R_1: the expected theoretical price (BS-implied expected price under R-measure); R_2: the expected excess return derived from the theory-implied expectation.

We further adopt a transfer formulation for European options under the R-measure:

$$\mathbb{E}_t[C_H J] = S_t e^{\mu(H-t)} \mathcal{N}(\hat{d}_1) - K e^{-r(T-H)} \mathcal{N}(\hat{d}_2), \tag{9}$$

$$\hat{d}_1 = \frac{\ln(S_t/K) + \mu(H-t) + r(T-H) + \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}}, \tag{10}$$

$$\hat{d}_2 = \frac{\ln(S_t/K) + \mu(H-t) + r(T-H) - \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}}. \tag{11}$$

These theory-guided R-factors are concatenated with the original variables to form the final input schema used by the forecasting module.

3.3. Risk-Aware Bayesian Context Layer (Optional)

To improve robustness under regime shifts, we optionally incorporate a risk-aware Bayesian classification layer inspired by **(author?)** [24, 25, 26]. In our architecture, this layer acts as a context estimator that outputs a risk state c_t (e.g., low/medium/high exposure) from context features such as volatility proxies, liquidity metrics, and geopolitical risk indicators. The risk state can be used in two deployment-friendly ways: (C1) Feature gating: re-weight features or select a reduced feature set conditioned on c_t. (C2) Model routing: select among model variants on the Pareto front (Section 3.5) depending on the current risk regime.

We implement a dynamic adaptation for risk-factor weights:

$$w_t(r_i) = w_{t-1}(r_i) \cdot \exp\left(\eta \cdot \frac{\partial L}{\partial w_{t-1}(r_i)}\right), \tag{12}$$

where $w_t(r_i)$ is the weight of risk factor r_i at time t, η is the learning rate, and L is the forecasting loss.

This layer improves interpretability (explicit risk context) and enhances maintainability (risk logic is encapsulated as a module that can be updated without retraining the entire forecasting backbone).

3.4. Forecasting Backbones and R-measure Integration

We consider five model families: gcForest (with its improved variant ADgcForest), RNN, LSTM, Transformer, and VQ-VAE. All models consume the same feature schema $[x_t^{\text{opt}}, x_t^{\text{macro}}, x_t^R]$ to ensure fair comparison in a deployable DSS setting.

3.4.1. ADgcForest with R-measure

The gcForest model contains (i) multi-grained scanning and (ii) a cascade forest. Multi-grained scanning applies sliding windows to create enhanced representations, while the cascade automatically determines depth using multiple layers of random forests and completely random forests, reducing manual hyper-parameter tuning.

We integrate the R-factors from Section 3.2 and apply Tri-Objective Optimization-based Single-Layer Pruning (TOOSLP) within each cascade layer. The pruning optimization uses NSGA-III with a chromosome encoding of tree subsets, and evaluates candidates via Accuracy, Independent Diversity, and Coupled Diversity. The final subset is chosen from the Pareto set (typically selecting the highest-accuracy candidate under the Pareto front).

Engineering pipeline used in ADgcForest (as reflected in Figure 1): (1) DBSCAN extracts cluster centers as additional features. (2) PCA reduces dimensionality for efficient learning. (3) R-factors are concatenated to the feature set. (4) TOOSLP/TOOCEP performs hierarchical post-pruning layer-by-layer.

This design enhances both predictive performance and deployability: pruning reduces storage/computation, and coupled diversity makes the pruning aware of the cascade structure, reducing inference time and footprint while preserving ensemble strength.

3.4.2. Transformer with R-measure

The Transformer replaces recurrence with attention, enabling parallel computation and strong long-range dependency modeling [27]. We incorporate R-factors by concatenating them with other variables and feeding them into the encoder input with positional encoding, reflected in Figure 2. Multi-head attention then models risk-return-aware time-series dependencies.

3.5. Multi-Objective Pruning, Model Selection, and System Maintainability

3.5.1. Tri-objective optimization for deployable models

To support DSS deployment, we treat resource constraints as first-class concerns. Our tri-objective pruning framework (TOOCEP/TOOSLP) produces a Pareto set of candidate models that trade off predictive strength and model compactness. In the gcForest setting, objectives include Accuracy, Independent Diversity, and Coupled Diversity (used as proxies for ensemble effectiveness under pruning). In deployment, we additionally profile each candidate for: inference latency (per request, batch=1 and batch>1 if applicable); model size (trees kept / parameters / storage footprint); memory footprint (peak RAM/VRAM). A DSS can then select a model variant based on its SLA (e.g., "< X ms latency" or "< Y MB model size") without retraining.

3.5.2. Module maintainability and extensibility

The architecture improves operational qualities in four ways: Schema-stable interface: models consume a consistent feature schema; new features (e.g., additional R-factors) are versioned in the feature store. Replaceable backbone: gcForest/Transformer/VQ-VAE are plug-compatible backbones; upgrades do not require rewriting downstream services. Offline/online separation: training and pruning run offline; inference stays lightweight and stable. Monitoring

and update: drift detection and periodic re-training/pruning can be scheduled independently; the Bayesian risk layer (Section 3.3) can be updated without full model retraining.

4. Experiments

4.1. Dataset and Feature Schema (R-Optfa)

Dataset. We construct a comprehensive feature dataset using historical Standard & Poor's 500 Index Options from 1998 to 2021. Raw option variables include expiration date (days to maturity), implied volatility, historical volatility, moneyness (stock price / strike price), implied premium, Black–Scholes–Merton (BS) theoretical price, risk-free interest rate, delta, and the stock closing price at contract signing. In addition, motivated by factor-model limitations for option returns, we include seven stock characteristics—Dividend–Price Ratio, Earnings–Price Ratio, Book–to–Market Ratio, Net Equity Expansion, Term Spread, Default Yield Spread, and Stock Variance—following the predictive option-return setting in **(author?)** [30]. From these variables, we further construct two theory-guided R-factors using the structured model in Section 3.2: (i) the BS-implied expected price and (ii) the expected excess return.

Preprocessing. To support stable deployment and reduce temporal leakage in a DSS pipeline, we apply standard preprocessing: normalization, missing value imputation, and extreme value replacement. The resulting dataset is referred to as R-Optfa.

Descriptive statistics. Tables 1–2 summarize the distribution of returns and key option characteristics for all options and for call/put subsets. These statistics provide empirical context for the system-level evaluation: the return distributions are heavy-tailed (high skewness/kurtosis), and option-related variables show wide ranges, reinforcing the need for robust models and resource-aware selection.

Table 1 reports central tendencies for returns and option characteristics. The realized excess return exhibits substantial dispersion, and the BS-implied expected return shows a wide range, implying a challenging low signal-to-noise environment for forecasting. Days to maturity and moneyness vary considerably across call/put subsets, motivating a deployable model that remains robust under heterogeneous regimes.

Table 2 highlights heavy tails in return-related variables (notably large skewness and kurtosis), which can destabilize models that are not robustly regularized. For a DSS/BI deployment, this motivates both theory-guided feature augmentation (R-factors) and resource-aware pruning to avoid overfitting while meeting latency constraints.

4.2. Evaluation Protocol and Metrics

Rolling/recursive validation. To mimic operational deployment and detect temporal drift, we use a recursive split that partitions R-Optfa into training, validation, and test subsets over time. This protocol reduces temporal leakage and supports evaluation across market regimes, aligning with the needs of maintainable DSS/BI forecasting services.

Metrics. We report predictive performance using R^2 and RMSE on training/validation/test splits. To align with the "resource-aware architecture" theme, we additionally report testing (inference) time measured on a consistent runtime environment (Google Colaboratory) as a lightweight proxy for latency. While absolute time values may vary across hardware, relative comparisons are informative for model selection under resource constraints.

In order to emulate deployment in decision-support information systems (DSS), we adopt a rolling/recursive evaluation protocol, where models are trained on historical windows, validated on recent data, and tested on subsequent periods to assess temporal stability and drift sensitivity (Figure 4).

4.3. Performance Comparison with gcForest Variants (Ablation)

This subsection evaluates whether the proposed system enhancements—tri-objective pruning (TOOCEP/TOOSLP) and R-factor augmentation—improve both accuracy and deployability.

Key findings (for DSS/BI deployment)

- ADgcForest vs. gcForest (no R-factors): test RMSE decreases from 0.2756 to 0.0606 ($\approx$ 78% reduction), and test R^2 increases from 0.9536 to 0.9978, indicating substantially improved generalization under the same protocol.
- Effect of R-factors (ADgcForest): adding R-factors further reduces test RMSE from 0.0606 to 0.0457 ($\approx$ 25% reduction) and increases test R^2 from 0.9978 to 0.9987, suggesting theory-guided features improve out-of-sample robustness.
- Latency proxy: ADgcForest improves testing time over gcForest by $\sim$ 0.15s (no R) and $\sim$ 0.12s (with R), supporting the claim of improved operational efficiency.

Table 3 reports an ablation study on gcForest-family models under the same rolling evaluation protocol. Compared with the baseline gcForest, the tri-objectively pruned ADgcForest substantially improves test accuracy and reduces error, while also slightly lowering inference time. Importantly, incorporating R-factors further strengthens out-of-sample performance, supporting the system design choice of combining theory-guided features with resource-aware pruning for DSS deployment.

We further quantify the impact of tri-objective pruning and R-factor augmentation on forecasting error by reporting the test-set RMSE for gcForest, ADgcForest without R-factors, and ADgcForest with R-factors (Figure 6).

4.4. Performance Comparison with State-of-the-Art Backbones (with R-factors)

We further benchmark representative sequence models under the same feature schema that includes R-factors, reflecting a realistic DSS scenario where a shared feature store feeds multiple model candidates.

Table 1
Summary Statistics for Options (Part 1) (Mean/Sd/Min/Q10/Q25/Median).

Panel	Variable	Mean	Sd	Min	Q10	Q25	Median
All Options (n=2,108,371)							
	Realized Excess Return	−0.23	1.15	−1.14	−1.02	−1.00	−0.99
	BS-implied Expected Return	0.86	3.14	−1.13	−1.00	−0.83	−0.13
	Days to Maturity	238.12	214.89	10.00	30.00	91.00	152.00
	Moneyness	1.01	0.13	0.41	0.86	0.94	1.01
	Implied Volatility	0.20	0.07	0.03	0.12	0.14	0.19
	Historical Volatility	0.17	0.09	0.02	0.11	0.11	0.15
	Delta	0.00	0.56	−0.90	−0.75	−0.50	−0.10
Call Options (n=1,041,048)							
	Realized Excess Return	0.04	1.22	−1.14	−1.02	−1.00	−0.20
	BS-implied Expected Return	1.69	3.99	−1.13	−0.98	−0.56	0.37
	Days to Maturity	234.43	212.65	10.00	30.00	91.00	152.00
	Moneyness	1.00	0.11	0.44	0.85	0.94	1.00
	Implied Volatility	0.20	0.07	0.05	0.12	0.14	0.19
	Historical Volatility	0.17	0.09	0.02	0.11	0.11	0.15
	Delta	0.51	0.24	0.10	0.15	0.30	0.50
Put Options (n=1,067,323)							
	Realized Excess Return	−0.49	1.01	−1.14	−1.02	−1.01	−0.38
	BS-implied Expected Return	0.05	1.65	−1.13	−1.00	−0.92	−0.51
	Days to Maturity	241.72	217.00	10.00	30.00	91.00	152.00
	Moneyness	1.01	0.13	0.51	0.86	0.95	1.01
	Implied Volatility	0.20	0.07	0.06	0.12	0.14	0.19
	Historical Volatility	0.17	0.09	0.02	0.11	0.11	0.15
	Delta	−0.50	0.24	−0.90	−0.85	−0.70	−0.50

Table 2
Summary Statistics for Options (Part 2) (Q75/Q90/Max/Skew/Kurt).

Panel	Variable	Q75	Q90	Max	Skew	Kurt
All Options						
	Realized Excess Return	0.35	1.30	5.85	1.86	3.94
	BS-implied Expected Return	1.10	3.49	26.95	3.86	19.46
	Days to Maturity	365.00	547.00	730.00	1.10	0.07
	Moneyness	1.06	1.14	3.05	0.49	3.95
	Implied Volatility	0.23	0.29	1.28	1.43	4.59
	Historical Volatility	0.21	0.27	1.37	2.45	10.76
	Delta	0.50	0.75	0.90	0.01	−1.37
Call Options						
	Realized Excess Return	0.70	1.59	5.85	1.41	2.37
	BS-implied Expected Return	2.02	5.77	26.95	3.03	10.86
	Days to Maturity	365.00	547.00	730.00	1.14	0.17
	Moneyness	1.07	1.13	1.97	0.62	2.57
	Implied Volatility	0.23	0.29	1.28	1.44	4.67
	Historical Volatility	0.21	0.27	1.37	2.47	11.01
	Delta	0.70	0.85	0.90	−0.04	−1.18
Put Options						
	Realized Excess Return	0.84	1.80	5.85	2.65	8.05
	BS-implied Expected Return	0.33	1.72	26.85	4.13	29.27
	Days to Maturity	365.00	547.00	730.00	1.07	−0.02
	Moneyness	1.07	1.16	3.05	0.84	3.84
	Implied Volatility	0.23	0.29	1.21	1.42	5.04
	Historical Volatility	0.21	0.27	1.37	2.42	10.82
	Delta	−0.30	−0.15	−0.10	0.01	−1.28

Table 3
Performance comparison with gcForests (deployability view).

Setting	Model	Train R^2	Train RMSE	Val R^2	Val RMSE	Test R^2	Test RMSE	Testing Time
No R-factor	gcForest	0.9981	0.0606	0.9786	0.1997	0.9536	0.2756	4.4759s
No R-factor	ADgcForest	0.9997	0.0239	0.9988	0.0467	0.9978	0.0606	4.3243s
With R-factor	ADgcForest	0.9998	0.0227	0.9992	0.0380	0.9987	0.0457	4.3552s

Table 4
Results with R-factors on R-Optfa (accuracy + latency proxy).

Model	Train R^2	Train RMSE	Val R^2	Val RMSE	Test R^2	Test RMSE	Testing Time
RNN	0.6579	0.6725	0.6553	0.6728	0.6567	0.6731	5.1224s
LSTM	0.7873	0.5303	0.7829	0.5339	0.7847	0.5330	4.2550s
Transformer	0.8245	0.5982	0.6757	0.7646	0.7989	0.6473	4.9634s
VQ-VAE	–	–	–	–	0.8534	0.4398	6.2377s
ADgcForest	0.9998	0.0227	0.9992	0.0380	0.9987	0.0457	4.3552s

Table 4 compares deep and ensemble backbones under the same R-factor-augmented feature schema. ADgcForest achieves the strongest accuracy across all splits, indicating superior robustness for high-dimensional option forecasting. Among neural baselines, VQ-VAE yields the lowest RMSE but incurs the highest testing time, which may become a bottleneck under strict SLA constraints. LSTM is the fastest among listed neural models, yet its error remains substantially larger than ADgcForest, illustrating the practical trade-off between accuracy and deployability in DSS settings.

From a system-integration perspective, model selection in DSS/BI should consider both predictive quality and inference cost. We therefore present a Pareto-style comparison across ensemble and deep backbones using test RMSE and testing time as a latency proxy (Figure 7), which supports SLA-aware deployment decisions (e.g., high-accuracy vs. low-latency regimes).

4.5. Deployment Implications for DSS/BI Architecture

From an information-system perspective, the results suggest the following deployment guidance: Default module choice. Under the current schema, ADgcForest provides the best accuracy with competitive latency, making it a strong default for production-like forecasting services. SLA-aware routing. If a DSS prioritizes latency over accuracy, LSTM may be chosen; however, its error gap suggests it should be reserved for coarse-grained screening tasks rather than precision pricing. Maintainability and evolution. The consistent feature schema (including R-factors) enables plug-and-play evaluation of new model backbones without reworking downstream interfaces; tri-objective pruning further supports model updates that satisfy memory/latency budgets.

5. Conclusion

This paper proposed a resource-aware forecasting architecture for decision-support information systems (DSS/BI) and evaluated it on option price prediction as a representative high-dimensional time-series application. Under a unified rolling/recursive protocol on the R-Optfa dataset, the TOOCEP-pruned ADgcForest achieved the strongest overall performance and deployability profile. Specifically, without R-factors, ADgcForest improved test performance from (R^2=0.9536) and RMSE (=0.2756) (gcForest) to (R^2=0.9978) and RMSE (=0.0606), while reducing testing time from 4.4759s to 4.3243s. With R-factors enabled, ADgcForest further reached (R^2=0.9987) and RMSE (=0.0457) on the test set with 4.3552s testing time, confirming that combining theory-guided R-measure features with tri-objective pruning yields a favorable accuracy–latency trade-off for deployment-oriented forecasting services.

Compared with state-of-the-art backbones using the same R-factor-augmented feature schema, ADgcForest remained dominant in predictive quality and operational balance: RNN obtained test (R^2=0.6567) and RMSE (=0.6731) (5.1224s), LSTM achieved (R^2=0.7847) and RMSE (=0.5330) (4.2550s), the optimized Transformer reached (R^2=0.7989) and RMSE (=0.6473) (4.9634s), and VQ-VAE delivered (R^2=0.8534) and RMSE (=0.4398) but incurred the highest testing time (6.2377s). These results indicate that, for DSS/BI integration, resource-aware pruning and feature refinement can shift models toward a better deployability frontier, enabling SLA-aware selection between high-accuracy and low-latency regimes.

Limitations and future work include extending evaluation beyond a single market and fixed feature schema (e.g., cross-market tests and regime-shift stress testing), replacing the current runtime-based latency proxy with finer-grained profiling (CPU/GPU breakdown, memory footprint, throughput under different batch sizes), and exploring dynamic feature selection and online updating to reduce maintenance overhead. We also plan to extend the architecture to additional instruments (e.g., futures and swaps) to support a broader class of data-driven forecasting services in real-world decision-support pipelines.

Acknowledgments

This work is not supported by any funding.

Declarations

Declaration of generative AI and AI-assisted technologies. The authors did not use generative AI or AI-assisted technologies in the writing of this manuscript.

CRediT authorship contribution statement. Rui Zhang: Conceptualization, Methodology, Software, Validation, Writing–original draft. Jiayi Li: Methodology, Software, Investigation. Binglin Guo: Investigation, Visualization. Chenxue Yang: Supervision, Resources, Writing–review & editing.

Data availability. The S&P 500 Index Options data used in this study are subject to data license restrictions. Aggregated statistics and code for preprocessing and model training can be made available upon reasonable request to the corresponding author.

Declaration of competing interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] F. Abdi, S. Abolmakarem and A.K. Yazdi. 2025. A Machine Learning-Based Approach for Multi-Objective Multi-Product Supply Chain Optimization via Demand Forecasting. *IEEE Access* 13 (2025), 3600–3615.

[2] H. Awijen, H. Ben Ameur and W. Louhichi. 2025. Forecasting oil price in times of crisis: New evidence from machine learning versus deep learning models. *Annals of Operations Research* 345, 2 (2025), 979–1002.

[3] Q. Bai, H. Chen, W. Li and B. Song. 2025. DeepForest-HTP: A novel deep forest approach for predicting high-dimensional financial sequences. *Computer Methods and Programs in Biomedicine* 258 (2025), 108514.

[4] J.K. Chauhan, T. Ahmed and A. Sinha. 2025. A novel deep learning model for stock market prediction using a sentiment analysis system. *Connection Science* 37, 1 (2025), 2455070.

[5] L. Chen and R. Gupta. 2026. Pareto-Efficient Deep Forest: Multi-Objective Learning for High-Frequency Option Prediction. In *Proceedings of the 2026 ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM.

[6] Q. Chen. 2025. Comparing Different Transformer Model Structures for Stock Prediction. *Expert Systems with Applications* 268 (2025), 120282.

[7] Y. Chen and J. Xu. 2026. Deep Learning for US Bond Yield Forecasting: An Enhanced LSTM–LagLasso Framework. *Journal of Intelligence and Engineering Technology* 1, 1 (2026), 1–12.

[8] Y. Dong, Z. Wu and Y. Hao. 2025. Dynamic fusion of multi-source heterogeneous data using MOE mechanism for stock prediction. *Applied Intelligence* 55, 6 (2025), 1–14.

[9] F. Ghallabi, B. Souissi, A. M. Du and S. Ali. 2025. ESG stock markets and clean energy prices prediction: Insights from advanced machine learning. *International Review of Financial Analysis* 97 (2025), 103889.

[10] S. Giantsidi and C. Tarantola. 2025. Deep learning for financial forecasting: A review of recent trends. *International Review of Economics & Finance* 104 (2025), 104719.

[11] A. Koochali, E. Tahaei, A. Dengel and S. Ahmed. 2025. VAEnue: A new avenue for VAE application on probabilistic forecasting. *Applied Intelligence* 55, 6 (2025), 1–23.

[12] P. Kumar and A. Singh. 2026. Transformer-based Multi-Grained Scanning for Volatility Forecasting. *ACM Transactions on Intelligent Systems and Technology* 17, 1 (2026), 1–22.

[13] D. Li, S. Huang, G. Wen and Z. Zhang. 2025. BrainEnsemble: A brain-inspired effective ensemble pruning algorithm for pattern classification. *Cognitive Computation* 17, 1 (2025), 1–21.

[14] S. Liu. 2026. Distributionally Robust Portfolio Optimization Based on Wasserstein Generative Modeling. In *ACM International Conference on Computing Frontiers (CF '26)*. ACM.

[15] Y. Liu. 2026. Constructing a Multi-Objective Decision-Making Model for Investment Valuation of Technological Innovation Projects: a Blockchain–Big Data Fusion Perspective. *Journal of Intelligence and Engineering Technology* 1, 1 (2026), 1–10.

[16] Z. Mao, X. Chen and C. Wu. 2025. Reinforced Distillation Learning: Fine-Grained Imbalanced Classifier for Financial Crisis Prediction. *Computational Economics* 52, 2 (2025), 1–34.

[17] A.U. Özsoy and Ö. Uğur. 2025. The QLBS Model within the presence of feedback loops through the impacts of a large trader. *Computational Economics* 53, 1 (2025), 1–28.

[18] J.M. Solleder, F. Silvy and M. Olarreaga. 2026. Protection for sale without aggregation bias in high-dimensional financial datasets. *Journal of International Economics* 160 (2026), 104222.

[19] A. Song, X. Jiang, M. Jiang and C. Wu. 2026. A Novel Hybrid Model to Forecast Crude Oil Price Based on Hybrid Kernel Extreme Learning Machine and Multi-Objective Chaos Game Optimization. *International Journal of Computational Intelligence Systems* 19, 1 (2026), 1184.

[20] H.G. Souto. 2025. Evaluating the Efficacy of NHITS for Forecasting Stock Realized Volatility: A Comparative Analysis with Established Models. *Computational Economics* 51, 3 (2025), 1–58.

[21] M. Steinbacher and M. Steinbacher. 2025. An Identification and Estimation of Stock Price Pattern Equations using K-Means and Neural Evolution. *Computational Economics* 54, 4 (2025), 1–44.

[22] T. T. Thach. 2025. Forecasting stock market indices using integration of encoder, decoder, and attention mechanism. *Entropy* 27, 1 (2025), 82.

[23] H. Wang and J. Cheng. 2026. STORM: A Spatio-Temporal Factor Model Based on Dual Vector Quantized Variational Autoencoder. *Computational Economics* 60, 1 (2026), 1–28.

[24] S. Wang and L. Zhao. 2025. Multi-Objective Bayesian Optimization for Hyperparameter Tuning in Financial Risk Models. *Journal of Computational Finance* 29, 2 (2025), 45–68.

[25] M. Yin and L.F. Frank. 2026. Generative Diffusion Models for Option Pricing: A Novel Framework for Modeling Volatility Dynamics in US Financial Markets. *Journal of Industrial Engineering and Applied Science* 3, 6 (2026), 23–29.

[26] X. Zeng, C. Liang, Q. Yang, F. Wang and J. Cai. 2025. Enhancing stock index prediction: A hybrid LSTM-PSO model for improved forecasting accuracy. *PLOS ONE* 20, 1 (2025), e0310296.

[27] R. Zhang, R. Zou and Y. Zhao. 2025. BA-Net: Bridge attention in deep neural networks for financial trend capture. *Expert Systems with Applications* 292 (2025), 128525.

[28] T. Zhang and X. Li. 2026. TimeMAR: Multi-Scale Autoregressive Modeling for Unconditional Time Series Generation in Financial Markets. In *Proceedings of the 2026 ACM International Conference on AI in Finance (ICAIF '26)*. ACM.

[29] Z. Zhang, Q. Liu, Y. Hu and H. Liu. 2025. Multi-feature stock price prediction by LSTM networks based on VMD and TMFG. *Journal of Big Data* 12, 1 (2025), 74.

[30] X. Zha. 2025. Using deep learning technology to predict the financial health status of enterprises in low-carbon economy. *International Journal of Low-Carbon Technologies* 20 (2025), 1488–1494.

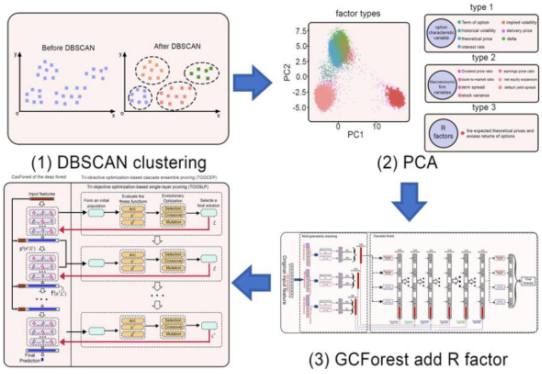

Figure 1: ADgcForest forecasting pipeline with R-measure: DBSCAN-based feature augmentation, PCA reduction, R-factor construction, and tri-objective pruning (TOO-CEP/TOOSLP) inside the cascade forest.

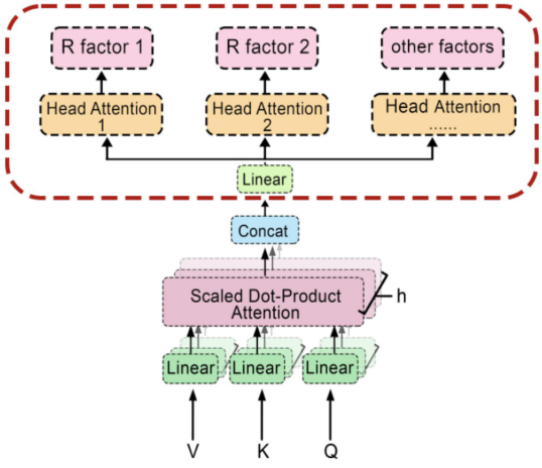

Figure 2: Transformer forecasting module augmented with R-factors: R-measure variables are concatenated with other features and consumed by multi-head attention for risk-return-aware time-series modeling.

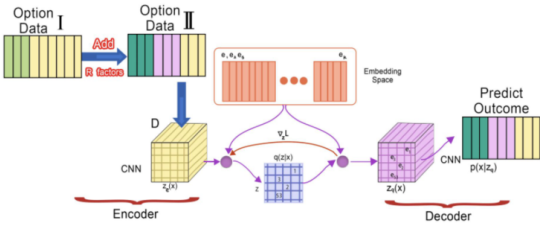

Figure 3: VQ-VAE-based representation learning for option forecasting with R-measure: risk-adjusted encoding, codebook refinement aligned with R-measure, and risk-sensitive decoding for robust reconstruction and prediction.

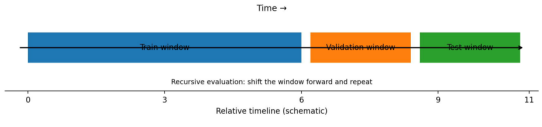

Figure 4: Rolling/recursive evaluation protocol that emulates DSS deployment: models are trained on past windows, validated on recent history, and tested on subsequent periods to assess temporal stability.

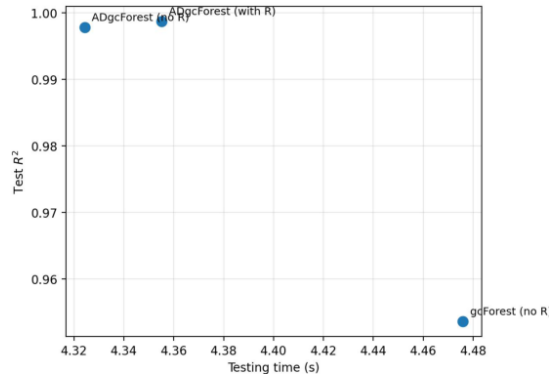

Figure 5: Accuracy–latency trade-off for gcForest variants on the test set: tri-objective pruning shifts the mode toward a better deployability frontier.

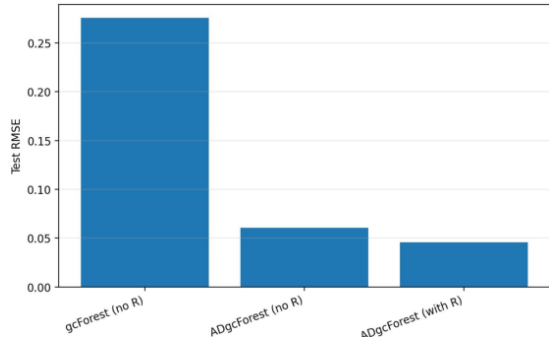

Figure 6: Impact of pruning and R-factor augmentation on forecasting error (test RMSE).

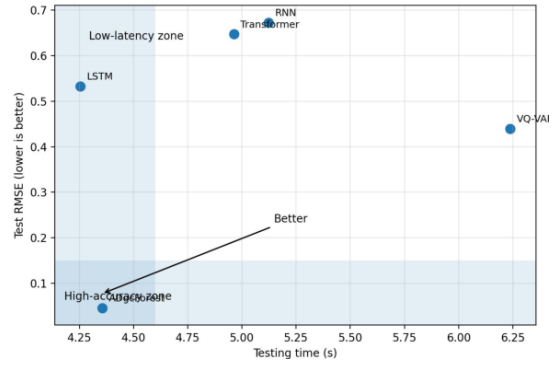

Figure 7: System-oriented model selection under resource constraints: Pareto comparison across ensemble and deep backbones using test accuracy and latency proxy.